

M18 - Portfolio Publication + Technical Defense

TURN PROJECT WORK INTO CREDIBLE JOB EVIDENCE

1. What a hiring person should see in 60 seconds

Visible item	What it should communicate
Repository title	business outcome, not just tool names
README opening	problem + trusted result
Architecture	source -> raw -> quality -> trusted -> serving + orchestration
Run section	exact reproducible steps
Quality	rules + quarantine/reconciliation
Failure evidence	one incident and recovery
Limitations	honest local/cloud/scale boundaries

2. What NOT to publish

- Real API keys/passwords/tokens or private connection strings.
- Employer/customer/private datasets.
- Huge raw extracts, .venv, caches, Docker volumes or generated warehouse files that make Git unusable.
- Screenshots that expose accounts/credentials/PII.
- A README claiming cloud production deployment when only a local plan was completed.

3. The 5-minute project defense

0:00-0:45 Business problem + sources + required result
0:45-1:30 Architecture and why each layer/tool exists
1:30-2:15 Trusted output grain + keys + quality/reconciliation
2:15-3:00 Idempotency, incremental, late data and backfill
3:00-3:45 One real failure: symptom -> root cause -> repair -> proof
3:45-4:30 Security/monitoring/testing
4:30-5:00 Limitation + what changes at 10x scale

4. Answer technical questions with evidence

Why did you choose this architecture?

Requirement -> simplest design that meets it -> rejected alternative -> evidence/limitation.

How do you prevent duplicates?

Business key + redelivery semantics + load/merge rule + rerun control results.

How do you know the numbers are right?

Source/accepted/quarantine/dedup reconciliation + total comparisons + tests.

What if yesterday is missing?

Detection -> bounded backfill -> same idempotent merge -> revalidation -> publish/freshness update.

Why Spark?

Data size/parallel workload/join/file problem that benefits; otherwise admit Python/SQL would be simpler.

Why Fabric?

Deployment target/business ecosystem + mapped services; separate actual runtime evidence from design plan.

What is your biggest limitation?

Specific limitation + consequence + next production improvement, not "none".

5. Clean-clone/publication checklist

Evidence gate - prove before move-on

- Clone or copy into a clean folder and follow README.
- No secrets found by manual/automated scan.
- Required safe sample data included or generation steps documented.
- Checker/tests run where environment supports them.
- Architecture/runbook/source contracts/quality rules visible.
- Evidence files small and understandable.
- Known limitations truthful.
- Commit history has meaningful milestones, not one giant final upload if avoidable.